Department of Electronic & Telecommunication Engineering
University of Moratuwa



EN3150 Pattern Recognition

Assignment 03: Simple Convolutional Neural Network to Perform Classification

Group 35 - Oasis

AROSHANA H.A.P.	230058N
BANDARA K.M.N.D.	230077V
CHANDRAKUMARA H.A.D.C.	230100M
NIRMANI W.T.	230444U

October 19, 2026

Contents

1 Resource-Constrained CNN for Edge Image Classification **2**
1.1 Introduction and Objective . . . 2
1.2 Data Preparation . . . 2

2 Custom Architecture Design **2**
2.1 Architectural Overview . . . 2
2.2 Model A Architectural Breakdown . . . 2
2.3 Model B Architectural Breakdown . . . 3
2.4 Hardware-Aware Activation Justifications . . . 3

3 Optimizer Selection & Tuning **4**
3.1 Optimizer Choice and Rationale . . . 4
3.2 Comparative Analysis with SGD Variants . . . 4
3.3 Justification of SGD Constant Flat Behavior . . . 4

4 Custom Model Training & Evaluation **5**
4.1 Training Dynamics . . . 5
4.2 Test Set Evaluation . . . 5
4.3 Trade-Off Discussion . . . 6

5 SOTA Fine-Tuning & Evaluation **6**
5.1 Selection of Lightweight SOTA Architectures . . . 6
5.2 Fine-Tuning and Performance Metrics . . . 6

6 Final Comparison & Trade-Off Discussion **7**
6.1 Custom Sub-100k Model vs. Pre-Trained SOTA Models . . . 7
6.2 Analysis of Trade-Offs . . . 7
6.3 Memory Footprint and Hardware Deployment Tier Compatibility . . . 8
6.4 Computational Cost, Latency, and Energy Consumption . . . 8
6.5 Classification Accuracy and Spatial Resolution Mismatch . . . 8

1 Resource-Constrained CNN for Edge Image Classification

1.1 Introduction and Objective

This assignment focuses on designing, training, and evaluating a lightweight image classifier using a convolutional neural network (CNN) tailored for embedded or low-resource edge devices. The complete source code and development history are available in the project's GitHub repository.

1.2 Data Preparation

In compliance with the assignment specifications, the **OCTMNIST** dataset from the MedMNIST collection was selected for low-resolution image classification. To emulate a resource-constrained edge sensor environment, all retinal optical coherence tomography (OCT) images are structured at a maximum resolution of 64×64 pixels in single-channel grayscale, ensuring no additional spatial downscaling or interpolation is required.

To eliminate data leakage caused by redundant scans across partitions, SHA-256 cryptographic pixel hashing was implemented. A total of 553 conflicting-label images were identified and excluded to prevent training corruption. The remaining valid samples were allocated into a duplicate-aware stratified split targeting the mandatory ratios:

- **Training Set:** 76,128 samples (70.00%)
- **Validation Set:** 15,170 deduplicated samples (15.00%)
- **Testing Set:** 15,172 deduplicated samples (15.00%)

A dedicated data pipeline was implemented to perform pixel normalization to the $[0.0, 1.0]$ range, channel expansion to a $(64, 64, 1)$ shape, buffer shuffling, and prefetching to optimize training throughput.

2 Custom Architecture Design

2.1 Architectural Overview

Both models process grayscale retinal OCT images of shape $64 \times 64 \times 1$ and classify them into 4 diagnostic categories (CNV, DME, drusen, normal). Model A uses standard 2D convolutions interleaved with max-pooling layers, flattening the final feature map into a dense classifier. Model B factorizes convolutions into Depthwise Separable Convolutions and replaces the high-parameter flattening layer with Global Average Pooling to strictly constrain the parameter count under 100,000 for edge hardware deployment.

2.2 Model A Architectural Breakdown

Model A interleaves standard 2D convolutions with max-pooling layers. The full layer-by-layer architectural parameter and Multiply-Accumulate (MAC) count breakdown for Model A is presented in Table 1.

Table 1: Layer-wise Architectural and Computational Details for Model A (Standard CNN)

Layer / Block	Type / Configuration	Output Shape	Params	MACs
Input	InputLayer	$(64, 64, 1)$	0	0
Layer 1	Conv2D (3×3, 32 filters, same)	$(64, 64, 32)$	320	1,179,648
Pool 1	MaxPooling2D (2×2, stride 2)	$(32, 32, 32)$	0	0
Layer 2	Conv2D (3×3, 64 filters, same)	$(32, 32, 64)$	18,496	18,874,368
Pool 2	MaxPooling2D (2×2, stride 2)	$(16, 16, 64)$	0	0
Layer 3	Conv2D (3×3, 128 filters, same)	$(16, 16, 128)$	73,856	18,874,368
Pool 3	MaxPooling2D (2×2, stride 2)	$(8, 8, 128)$	0	0
Flatten	Flatten	$(8192,)$	0	0
Dense 1	Dense (64 units, ReLU)	$(64,)$	524,352	524,288
Dropout	Dropout (rate = 0.3)	$(64,)$	0	0
Output	Dense (4 units, Softmax)	$(4,)$	260	256
TOTAL	-		**617,284**	**39,452,928**

2.3 Model B Architectural Breakdown

Model B utilizes Depthwise Separable Convolutions paired with Global Average Pooling to drastically minimize hardware complexity. The full breakdown for Model B is presented in Table 2.

Table 2: Layer-wise Architectural and Computational Details for Model B (Lightweight CNN)

Layer / Block	**Type / Configuration**	**Output Shape**	**Params**	**MACs**
Input	InputLayer	$(64, 64, 1)$	0	0
Layer 1	SeparableConv2D (3×3, 32 filters)	$(64, 64, 32)$	73	167,936
Pool 1	MaxPooling2D (2×2, stride 2)	$(32, 32, 32)$	0	0
Layer 2	SeparableConv2D (3×3, 64 filters)	$(32, 32, 64)$	2,400	2,392,064
Pool 2	MaxPooling2D (2×2, stride 2)	$(16, 16, 64)$	0	0
Layer 3	SeparableConv2D (3×3, 128 filters)	$(16, 16, 128)$	8,896	2,244,608
Pool 3	MaxPooling2D (2×2, stride 2)	$(8, 8, 128)$	0	0
GAP	GlobalAveragePooling2D	$(128,)$	0	0
Dense 1	Dense (64 units, ReLU)	$(64,)$	8,256	8,192
Dropout	Dropout (rate = 0.3)	$(64,)$	0	0
Output	Dense (4 units, Softmax)	$(4,)$	260	256
TOTAL	-		**19,885**	**4,813,056**

Comparative Efficiency: Model B achieves a **96.78% parameter reduction** (19,885 vs. 617,284 parameters, well beneath the 100,000 ceiling) and an **87.80% reduction in MAC operations** (4,813,056 vs. 39,452,928 MACs) compared to Model A.

2.4 Hardware-Aware Activation Justifications

- **Hidden Layers (ReLU):** ReLU ($\max(0, x)$) was chosen for all hidden layers to optimize execution speed and power efficiency on resource-constrained edge hardware through three main mechanisms:
  - *Elimination of Complex Math:* Functions like Sigmoid or Tanh require expensive exponential (expf) and division operations. Without dedicated hardware accelerators, these take 20 to 40+ clock cycles per neuron via software approximations. ReLU uses a single conditional sign-bit check or comparator instruction, executing in just 1 clock cycle.
  - *INT8 Quantization Efficiency:* Because ReLU is piecewise linear, it operates directly on quantized 8-bit integer (INT8) accumulators by simply clamping values below the zero-point, completely avoiding the need for memory-heavy Look-Up Tables (LUTs) or dequantization steps.
  - *Feature Sparsity:* By setting negative values to zero, ReLU creates sparse feature maps (many zero values), which reduces memory bus switching power and allows hardware zero-skipping logic to bypass unnecessary multiplications.
- **Output Layer (Softmax):** The Softmax function ($\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$) is used exclusively at the final output layer for classification, keeping its computational impact minimal for two reasons:
  - *Localized Exponentials:* Exponentials are restricted only to the 4 final output logits rather than thousands of hidden feature map elements, making the overhead negligible.
  - *Bypassing during Inference:* Because exponentiation is strictly monotonic (preserving order: if $a > b$, then $e^a > e^b$), edge hardware can entirely skip the Softmax calculation during real-time inference. The predicted class is determined directly by finding the highest raw logit: $\arg\max_i(\text{Softmax}(\mathbf{z})) \equiv \arg\max_i(z_i)$.

3 Optimizer Selection & Tuning

3.1 Optimizer Choice and Rationale

The **Adam (Adaptive Moment Estimation)** optimizer with an initial learning rate of $\eta = 0.001$ ($\beta_1 = 0.9, \beta_2 = 0.999$) was selected as the primary optimizer for training Model B. The rationale for this selection includes:

1. **Per-Parameter Adaptive Learning Rates for Depthwise Separable Convolutions:** In Model B, factorized depthwise filters (3×3 spatial kernels operating per channel) and pointwise filters (1×1 cross-channel projections) exhibit markedly different gradient magnitudes and curvature scales. A single global scalar learning rate in vanilla SGD updates depthwise and pointwise weights sub-optimally. Adam normalizes each parameter's update by the root-mean-square of its historical squared gradients, automatically balancing updates across heterogeneous layer types.

2. **Robustness to Class Imbalance:** The OCTMNIST dataset exhibits significant class imbalance (e.g., normal vs. minority classes like drusen and DME). Minority classes produce sparser, higher-variance mini-batch gradients. Adam's second-moment scaling amplifies infrequent but informative gradient signals for minority classes while preventing majority-class dominance in early epochs.

3.2 Comparative Analysis with SGD Variants

To rigorously evaluate convergence, Model B was trained across 10 epochs under identical initialization using three distinct configurations:

1. **Adam ($\eta = 0.001$):** Selected optimizer, yielding smooth, rapid convergence and reaching a peak validation accuracy of **89.26%** (final validation loss of 0.3186).

2. **Standard Stochastic Gradient Descent (SGD) ($\eta = 0.01, \textbf{momentum} = 0.0$):** Prone to slow convergence, getting stuck in ravines, and failing to break past initial plateaus on complex architectures without adaptive scaling.

3. **SGD with Momentum ($\eta = 0.01, \textbf{momentum} = 0.9$):** Incorporates an exponentially weighted moving average of past gradients to accelerate descent in consistent directions and dampen oscillations.

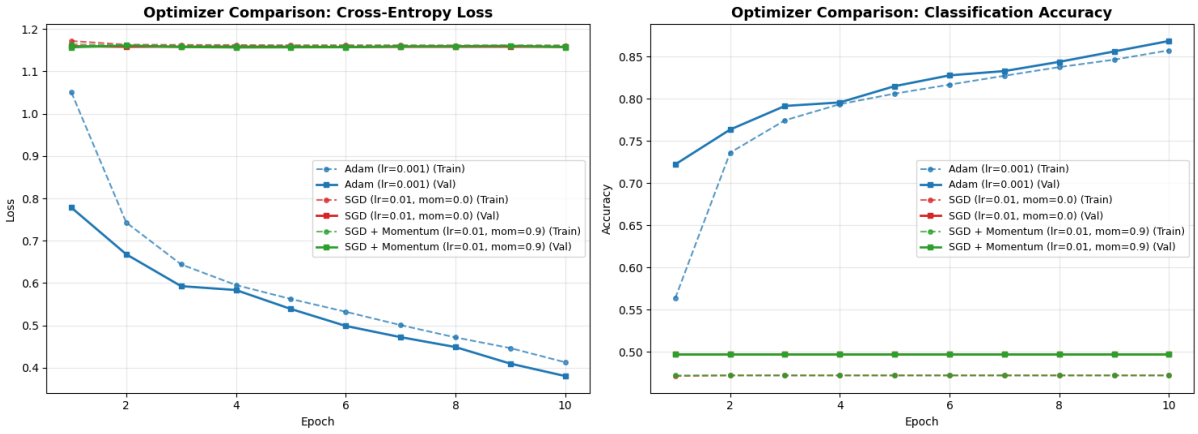


Figure 1: Empirical comparison of Adam ($\eta = 0.001$), Standard SGD ($\eta = 0.01, \text{mom} = 0.0$), and SGD with Momentum ($\eta = 0.01, \text{mom} = 0.9$) showing cross-entropy loss (left) and classification accuracy (right) over 10 epochs.

3.3 Justification of SGD Constant Flat Behavior

As illustrated in Figure 1, both **Standard SGD** and **SGD with Momentum** exhibit a completely flat, constant behavior stagnating at a cross entropy loss of ~ 1.16 and a classification accuracy of $\sim 49.74\%$ across all 10 epochs. This failure to learn is attributed to the following factors:

- **Gradient Scale Disparity in Depthwise Separable Convolutions:** Depthwise separable convolutional layers combined with Global Average Pooling produce highly heterogeneous gradient scales across layers. A fixed global learning rate ($\eta = 0.01$) is either excessively large (causing destabilization or numerical saturation in softmax) or too small for the initial layers, resulting in vanishing gradient updates.
- **Trapping in Saddle Points and Class Priors:** Without per-parameter adaptive scaling, SGD and momentum-based SGD are unable to dynamically adjust step sizes in ill-conditioned loss landscapes. Consequently, the network remains trapped near the initial random guessing state corresponding to the majority-class distribution prior.
- **Impact of Momentum Parameter ($\gamma = 0.9$):** While momentum ($\gamma = 0.9$) mathematically accumulates velocity $v_t = \gamma v_{t-1} + \eta \nabla_{\mathcal{L}}(\theta_t)$ to smooth trajectories and overcome local minima, it cannot rescue training if the instantaneous gradients $\nabla_{\mathcal{L}}(\theta_t)$ remain uniformly near-zero or miscalibrated across layers. Adam overcomes this by decoupling step sizes from gradient magnitudes via second-moment normalization.

4 Custom Model Training & Evaluation

4.1 Training Dynamics

Both models, Model A (Standard CNN, containing 617,284 parameters) and Model B (Lightweight Depthwise Separable CNN, containing 19,885 parameters) were trained for 20 epochs on the OCTMNIST dataset using the Adam optimizer with a learning rate of 0.001. Training and validation loss and accuracy curves were plotted across all epochs to monitor convergence dynamics and prevent overfitting.

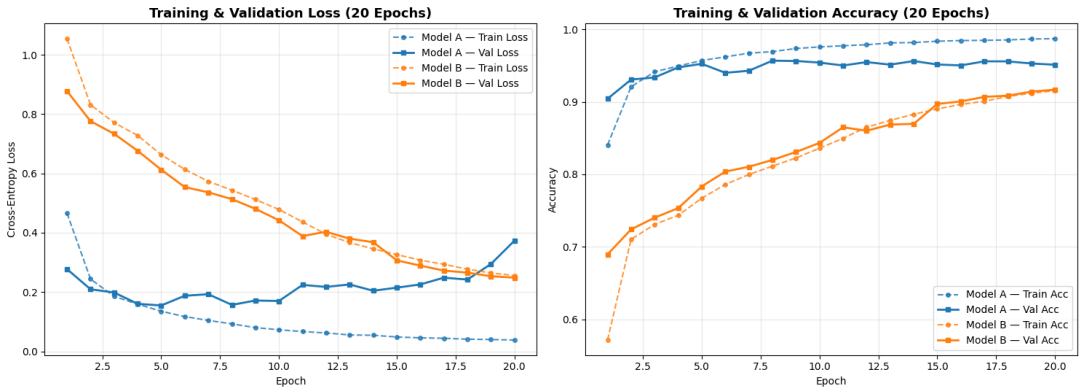


Figure 2: Training and validation loss and accuracy curves for Model A and Model B over 20 epochs.

As observed from the loss curves, Model A exhibits signs of overfitting starting around epoch 5, where the validation loss begins to diverge and gradually increase (reaching up to ~ 0.37 by epoch 20) despite the training loss continuing to decrease steadily toward near-zero. This phenomenon could be attributed to Model A's higher parameter capacity ($617,284$ parameters), which potentially allows it to memorize noise and fine-grained idiosyncrasies in the training distribution rather than learning fully generalized feature representations. In contrast, Model B demonstrates stable convergence without noticeable overfitting, which may be attributed to its strong structural regularization and extreme parameter constraint ($19,885$ parameters). To mitigate Model A's validation loss divergence in future iterations, standard regularization strategies could be implemented—such as introducing early stopping around epoch 5 where validation loss reaches its optimal minimum (~ 0.15), applying heavier dropout, or incorporating weight decay.

4.2 Test Set Evaluation

Both models were rigorously evaluated on the deduplicated test set comprising 15,172 images. Performance metrics including test accuracy, macro precision, macro recall, macro F1-score, and confusion

matrices were recorded. Model A achieved a test accuracy of 95.06% with a macro F1-score of 91.22%, whereas Model B achieved a test accuracy of 91.42% with a macro F1-score of 83.13%.

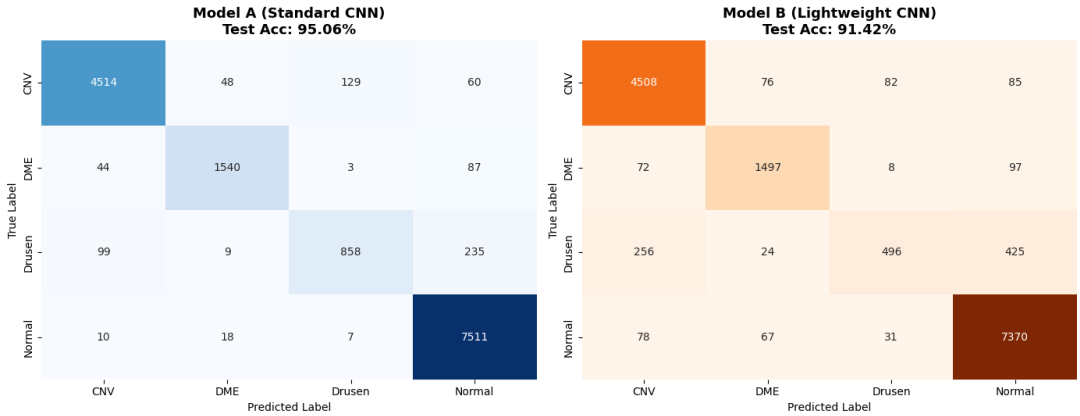


Figure 3: Confusion matrices evaluated on the test set for Model A and Model B.

Table 3: Comparison Between Custom CNN Models (Model A vs. Model B)

Model	Total Params	Size (KB)	Time/Epoch (s)	Test Acc. (%)
Model A (Standard CNN)	617,284	7,280.41	100.32	95.06%
Model B (Lightweight CNN)	19,885	283.66	64.66	91.42%

4.3 Trade-Off Discussion

Transitioning from standard convolutions to depthwise separable convolutions yields an extraordinary reduction in parameter count (from 617,284 down to 19,885, representing a $\approx 96.8\%$ decrease) and estimated model size on disk (from 7,280.41 KB to 283.66 KB). This lightweight design significantly decreases memory footprint and computational overhead while incurring only a minor, acceptable decrease in test classification accuracy (from 95.06% to 91.42%). Such a trade-off demonstrates the high efficiency of depthwise separable convolutions, making them exceptionally well-suited for resource-constrained edge hardware and real-time clinical inference applications.

5 SOTA Fine-Tuning & Evaluation

5.1 Selection of Lightweight SOTA Architectures

Two pre-trained lightweight architectures designed for mobile and edge computing were selected:

1. **MobileNetV2**: Designed specifically for mobile and embedded vision applications, utilizing inverted residual blocks with linear bottlenecks and depthwise separable convolutions.

2. **EfficientNet-B0**: Developed via neural architecture search optimizing accuracy and computational efficiency through compound scaling of network depth, width, and resolution, augmented with squeeze-and-excitation channel-attention gates.

5.2 Fine-Tuning and Performance Metrics

Both models were fine-tuned using the identical dataset splits as the custom networks. Performance metrics on the test set were evaluated, noting the exact parameter counts and total model sizes in megabytes (MB).

Table 4: Performance and Complexity Comparison of SOTA Lightweight Models

Architecture	Total Params	FP32 Size	Saved Size	Test Acc. (%)	Macro F1 (%)
MobileNetV2	2,263,108	8.63 MB	23.48 MB	92.56%	86.52%
EfficientNet-B0	4,054,695	15.47 MB	41.46 MB	93.42%	88.30%

MobileNetV2 contains $2,263,108$ parameters with an FP32 weight size of 8.63 MB (and a saved model size of 23.48 MB), achieving a test accuracy of 92.56% and a macro F1-score of 86.52%. EfficientNet-B0 contains $4,054,695$ parameters with an FP32 weight size of 15.47 MB (and a saved model size of 41.46 MB), yielding a test accuracy of 93.42% and a macro F1-score of 88.30%.

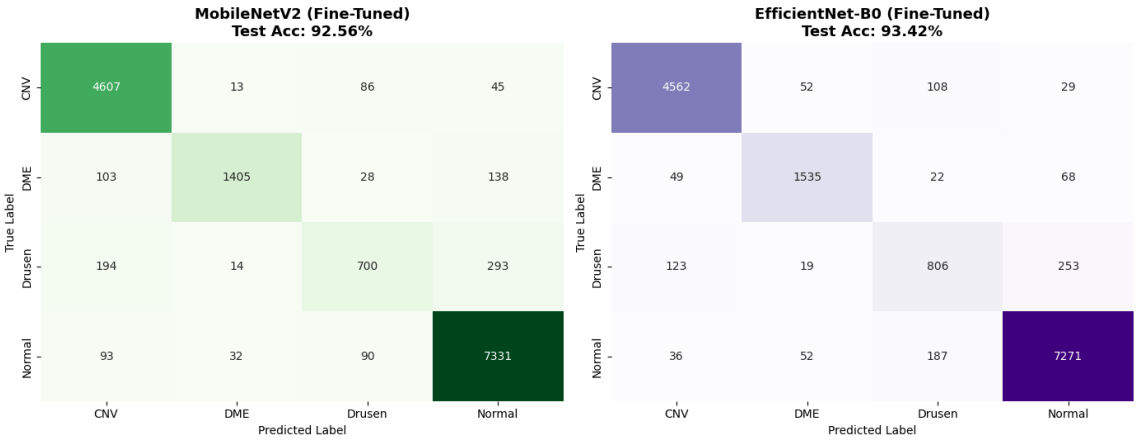


Figure 4: Confusion matrices evaluated on the test set for fine-tuned MobileNetV2 and EfficientNet-B0 architectures.

6 Final Comparison & Trade-Off Discussion

6.1 Custom Sub-100k Model vs. Pre-Trained SOTA Models

A comprehensive comparison was performed between our custom resource-constrained CNN (Model B) and the fine-tuned lightweight SOTA models.

6.2 Analysis of Trade-Offs

- **Accuracy vs. Footprint:** SOTA pre trained models achieve higher accuracy by leveraging generalized feature extractors learned from massive datasets, but at the cost of larger memory footprints and higher computational complexity.
- **Edge Suitability:** Custom sub 100k models provide precise control over memory, power, and storage budgets, making them superior for extreme microcontroller environments where SOTA models exceed physical hardware constraints.

Table 5: Comprehensive Comparison Across Custom and SOTA Architectures

Architecture	Category	Total Params	FP32 Size	Approx. MACs	Test Acc. (%)	Macro F1 (%)
Model B (Custom Lightweight)	Custom (< 100k)	19,885	0.076 MB	4.81 M	91.42%	83.13%
Model A (Custom Standard)	Custom Standard	617,284	2.35 MB	39.45 M	95.06%	91.22%
MobileNetV2 (Fine-Tuned)	Lightweight SOTA	2,263,108	8.63 MB	~25.6 M	92.56%	86.52%
EfficientNet-B0 (Fine-Tuned)	Lightweight SOTA	4,054,695	15.47 MB	~33.1 M	93.42%	88.30%

6.3 Memory Footprint and Hardware Deployment Tier Compatibility

- **Custom Model B (**$19,885$ **parameters,** ≈ 0.076 **MB FP32 /** ≈ 0.019 **MB INT8):** Requires less than 78 KB in 32-bit floating-point format and under 20 KB when quantized to 8-bit integers. It fits directly inside the on-chip Flash and SRAM of ultra-low-power microcontrollers such as ARM Cortex-M4/M7 or ESP32-S3 with zero external DRAM required.

- **MobileNetV2 (**2.26**M parameters,** 8.63 **MB FP32) & EfficientNet-B0 (**4.05**M parameters,** 15.47 **MB FP32):** MobileNetV2 and EfficientNet-B0 are $114\times$ and $204\times$ larger in parameter count than Model B. They target higher tier edge microprocessors such as Raspberry Pi, NVIDIA Jetson Nano, Google Coral Edge TPU, or mobile SoCs with hundreds of megabytes of RAM.

6.4 Computational Cost, Latency, and Energy Consumption

- **Multiply-Accumulate (MAC) Operations:** Model B processes single-channel $(64 \times 64 \times 1)$ grayscale OCT images in 4.81M MACs. Pre-trained ImageNet backbones require duplicating the single grayscale channel into a 3-channel tensor $(64 \times 64 \times 3)$, executing $5.3\times$ to $6.9\times$ more MACs ($\sim$ 25.6M to 33.1M MACs).

- **Activation Complexity:** Model B utilizes hardware-friendly 'ReLU' activations and sequential spatial downsampling. EfficientNet-B0 employs Swish ('SiLU') and Squeeze-and-Excitation ('Sigmoid') gates in every 'MBConv' block, introducing latency overhead on embedded CPUs without dedicated transcendental acceleration.

6.5 Classification Accuracy and Spatial Resolution Mismatch

- **SOTA Inductive Bias:** Pre-training on large-scale image datasets provides rich multi-scale edge and texture filters, enabling rapid feature transfer and high peak accuracy.

- **Spatial Resolution Trade-Offs:** MobileNetV2 and EfficientNet-B0 are architected for 224×224 inputs with five stride-2 downsampling stages, collapsing 64×64 inputs to a 2×2 feature map before global pooling. Subtle localized retinal lesions suffer severe spatial aliasing. In contrast, Model B preserves an 8×8 spatial grid via three pooling stages, maintaining higher fidelity for domain-specific medical images.